Academic Research Project

Will the Waymo stop if you throw a [tennis ball / skateboard / chair] at it?

input layer

hidden layers

output layer







tennis ball
skateboard
chair



0 (Waymo hit object)
1 (Waymo stopped)









CS 1671 / CS 2071
Human Language Technologies

Session 14: Neural networks part 2

Michael Miller Yoder

October 8, 2026



Assessments: project

- Project deliverable 1 is due **11:59pm tonight, Thu Oct 8**
  - Please fill out peer review: https://forms.cloud.microsoft/r/NKimWCuwyb

Assessments: project

- Project proposal presentations will be in class **next Tue Oct 13**
  - Add your group's slides to this shared presentation
  - Maximum 5 minutes, please

Project components

	Baseline system	Advanced system
Deliverable 1	Propose method	
Deliverable 2	Initial result	Propose method
Final report	Final results	Final results

SCI open-source LLMs available for API use:
Gemma 3, Llama 3.1, Deepseek R1

Assessments: homework

- Homework 2 on text classification is due **next Thu Oct 15**
  - Implement a logistic regression classifier with different features

Student-led study group

- A study group led by students has formed
- Let me know if you’re interested and I can connect you!

Review: neural networks

Discuss with a neighbor:

1. Describe the steps of computation that occur in the “forward pass” of a neural network to make predictions
2. How are word vectors like word2vec trained to roughly represent word meaning?

Overview: neural networks part 2

- Feedforward neural networks as classifiers
- Feedforward neural networks with word embedding input
- Training neural networks
- Coding activity: build an FFNN

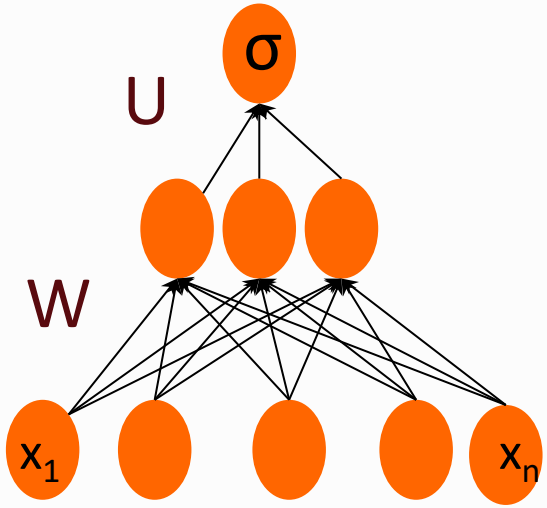
Feedforward neural nets as classifiers

Classification: Sentiment Analysis

We could do exactly what we did with logistic regression

Input layer are binary features as before

Output layer is 0 or 1



Slide adapted from Jurafsky & Martin

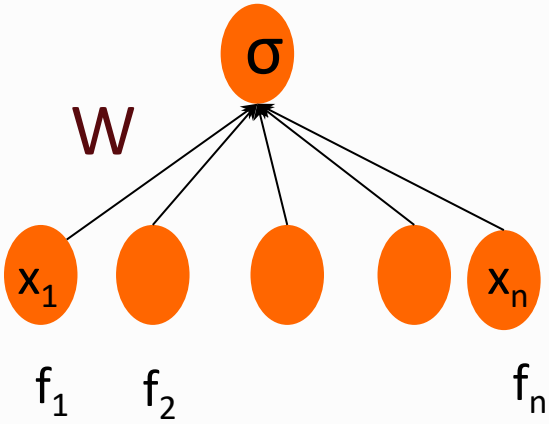
Sentiment Features

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

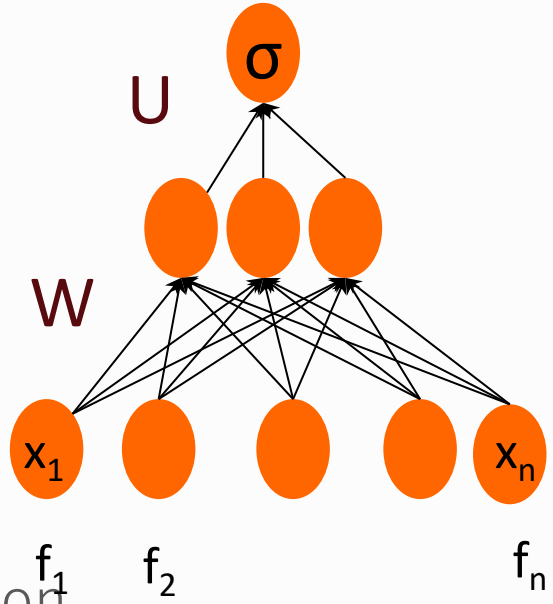
Slide adapted from Jurafsky & Martin

Feedforward nets for simple classification

Logistic Regression



2-layer feedforward network

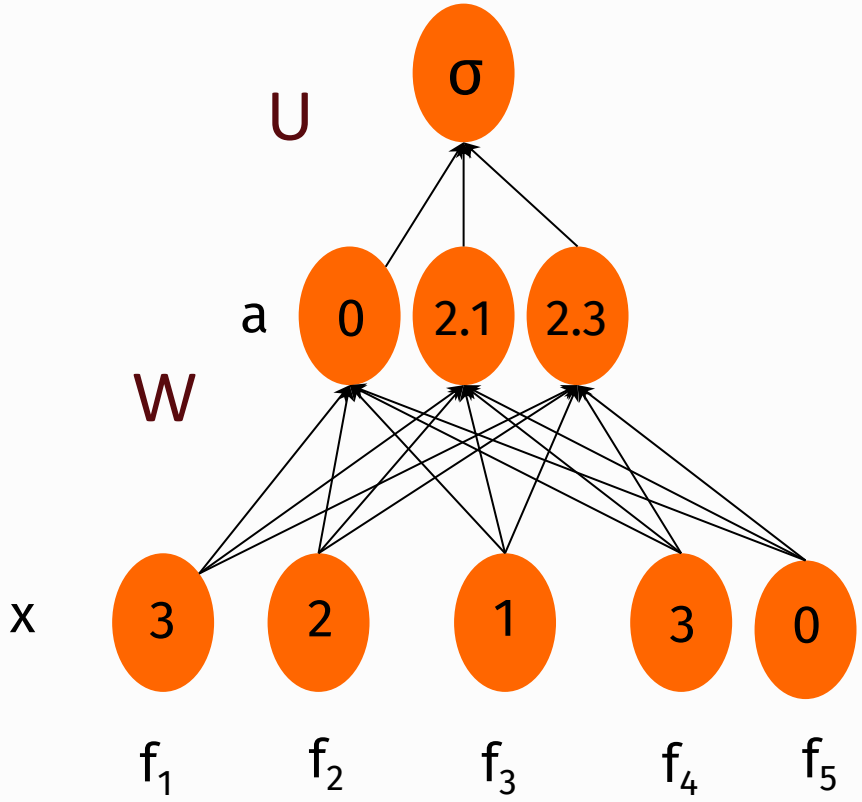


Just adding a hidden layer to logistic regression

- allows the network to use non-linear interactions between features
- which may (or may not) improve performance.

Slide adapted from Jurafsky & Martin

Working through an example



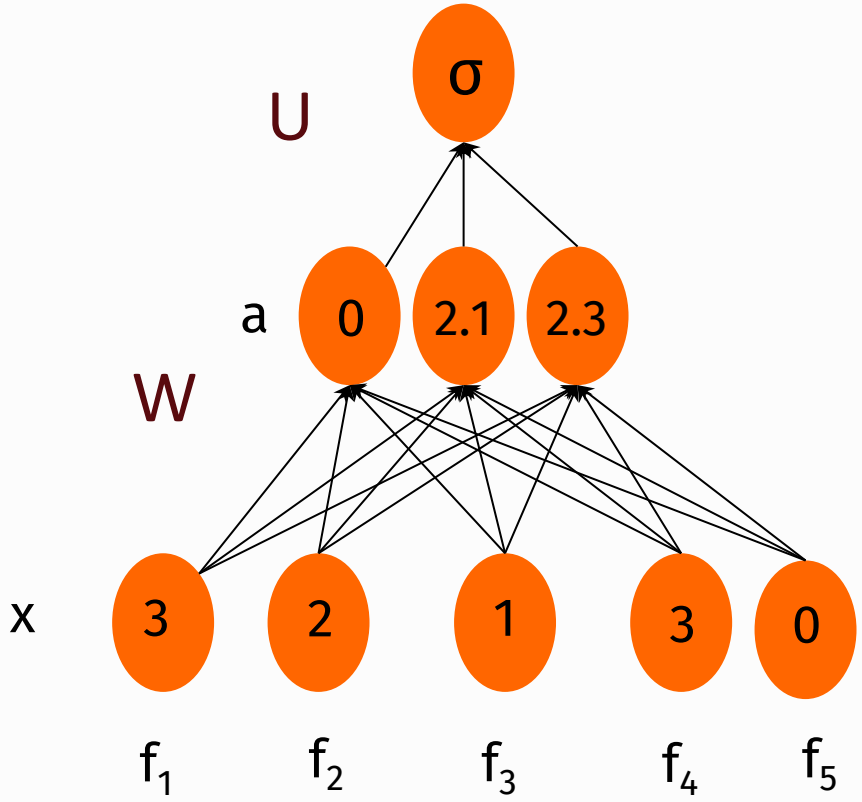
Input vector x:
[3 2 1 3 0]

W =
[0.3 1.0 -2.7
1.5 -2.2 0.9
2.5 -1.1 3.5
-5.2 0.2 0.7
4.3 1.2 -3.8]
b = 3

a = ReLU(**W**x + b)
a = ReLU([-6.2 2.1 2.3]
a = [0 2.1 2.3]

Slide adapted from Jurafsky & Martin

Working through an example



a = [0 2.1 2.3]

U =
[-4.2 3.8 0.4]
b = 1.2

y = σ(Ua + b)
y = σ(10.1)
y = 0.99

Slide adapted from Jurafsky & Martin

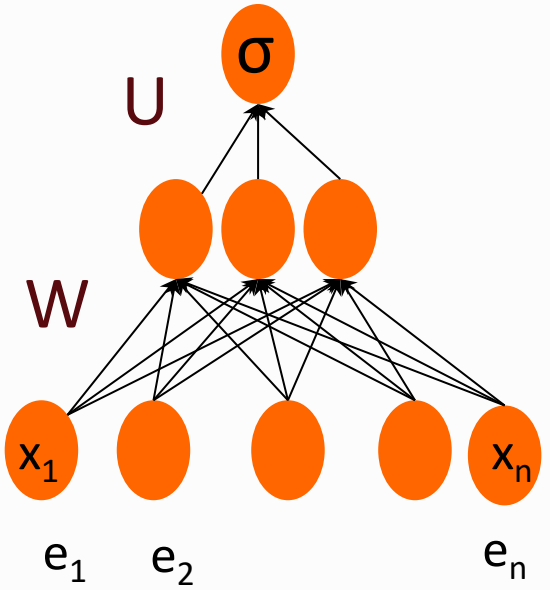
Feedforward neural networks with word embedding input

Even better: representation learning

The real power of deep learning comes from the ability to **learn** features from the data

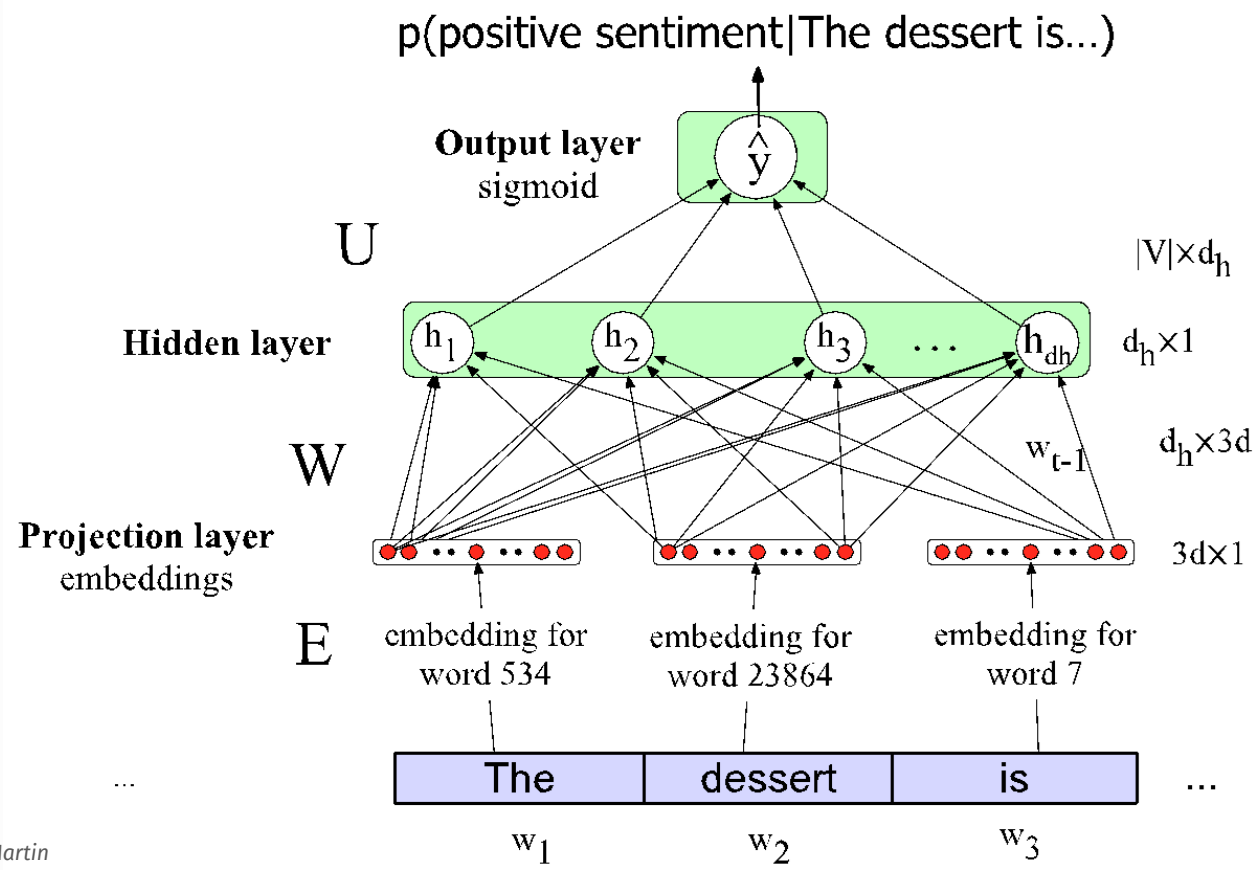
Instead of using hand-built human-engineered features for classification

Use learned representations like embeddings!



Slide adapted from Jurafsky & Martin

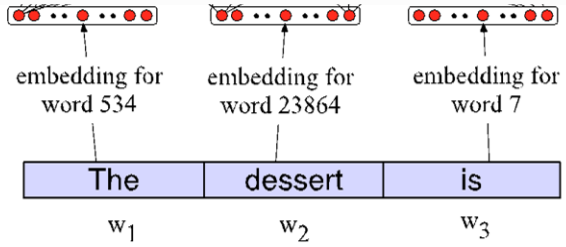
Neural net classification with embeddings as input features!



Slide adapted from Jurafsky & Martin

Issue: texts come in different sizes

This assumes a fixed size length (3)!
Kind of unrealistic.
Some simple solutions:

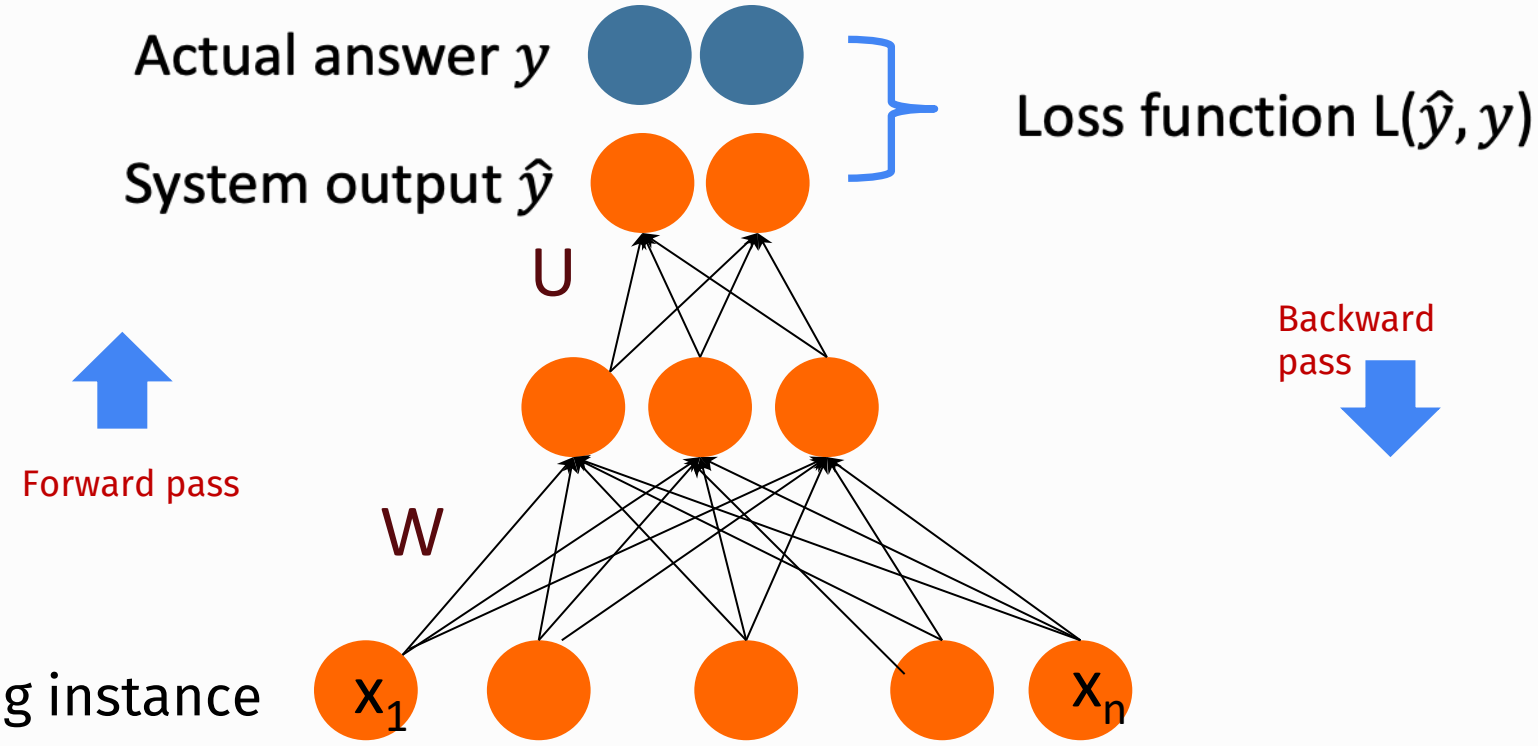


1. Make the input the length of the longest review
   - If shorter then pad with zero embeddings
   - Truncate if you get longer reviews at test time
2. Create a single "sentence embedding" (the same dimensionality as a word) to represent all the words
   - Take the mean of all the word embeddings
   - Take the element-wise max of all the word embeddings
     - For each dimension, pick the max value from all words

Slide adapted from Jurafsky & Martin

Training feedforward neural networks

Intuition: training a 2-layer Network



Slide adapted from Jurafsky & Martin

Remember stochastic gradient descent from the logistic regression lecture—find gradient and optimize

Slide credit: David Mortensen

The Intuition Behind Training a 2-Layer Network

For every training tuple (x, y)

1. Run **forward** computation to find the estimate $\hat{y}$
2. Run **backward** computation to update weights
    - For every output node
        - Compute the loss L between true y and estimated $\hat{y}$
        - For every weight w from the hidden layer to the output layer: update the weights
    - For every hidden node
        - Assess how much blame it deserves for the current answer
        - From every weight w from the input layer to the hidden layer
        - Update the weight

Slide credit: David Mortensen

Computing the gradient requires finding the derivative of the loss with respect to each weight in every layer of the network.

Error backpropagation through computation graphs.

Slide credit: David Mortensen

Reminder: gradient descent for weight updates

Use the derivative of the loss function with respect to weights $\frac{d}{dw} L(f(x;w),y)$

To tell us how to adjust weights for each training item

- Move them in the opposite direction of the gradient

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{d}{dw} L_{CE}(f(\mathbf{x};\mathbf{w}),y)$$

- For logistic regression

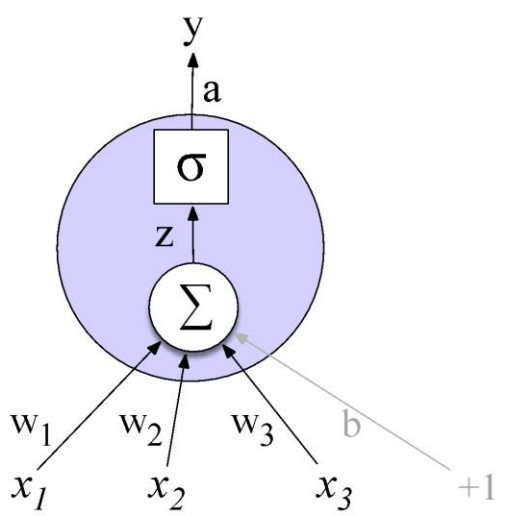
$$\frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

Slide adapted from Jurafsky & Martin

Where did that derivative come from?

Using the chain rule of derivatives! $f(x) = u(v(x))$

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$



Derivative of the weighted sum

Derivative of the Activation

Derivative of the Loss

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

Slide adapted from Jurafsky & Martin

How can I find that gradient for every weight in the network?

These derivatives on the prior slide only give the updates for one weight layer: the last one!

What about deeper networks? For training, we need the derivative of the loss with respect to each weight in every layer of the network

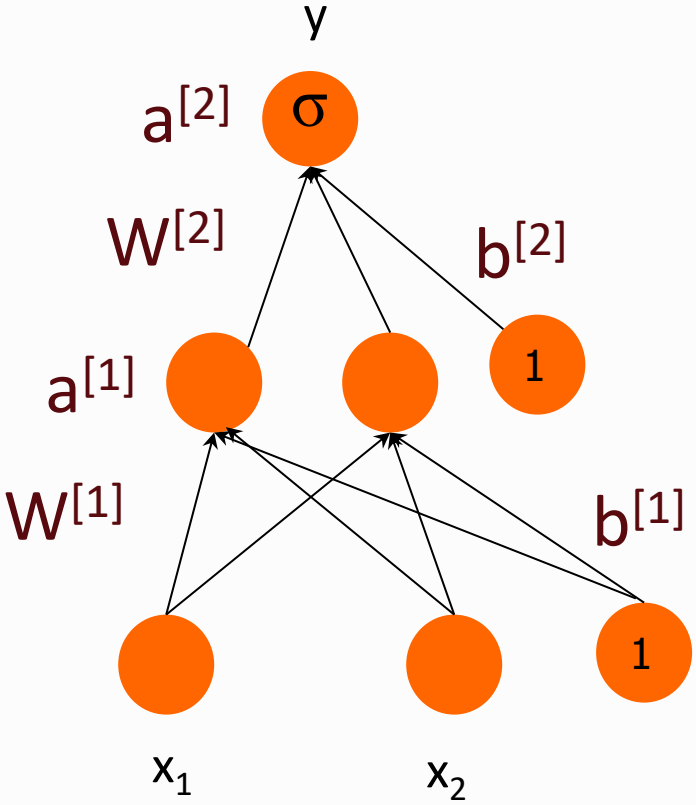
- Lots of layers, different activation functions?
- But the loss is computed only at the very end of the network!

Solution:

- Even more use of the chain rule!!
- This process is called **error backpropagation** (Rumelhart et al 1986)

Slide adapted from Jurafsky & Martin

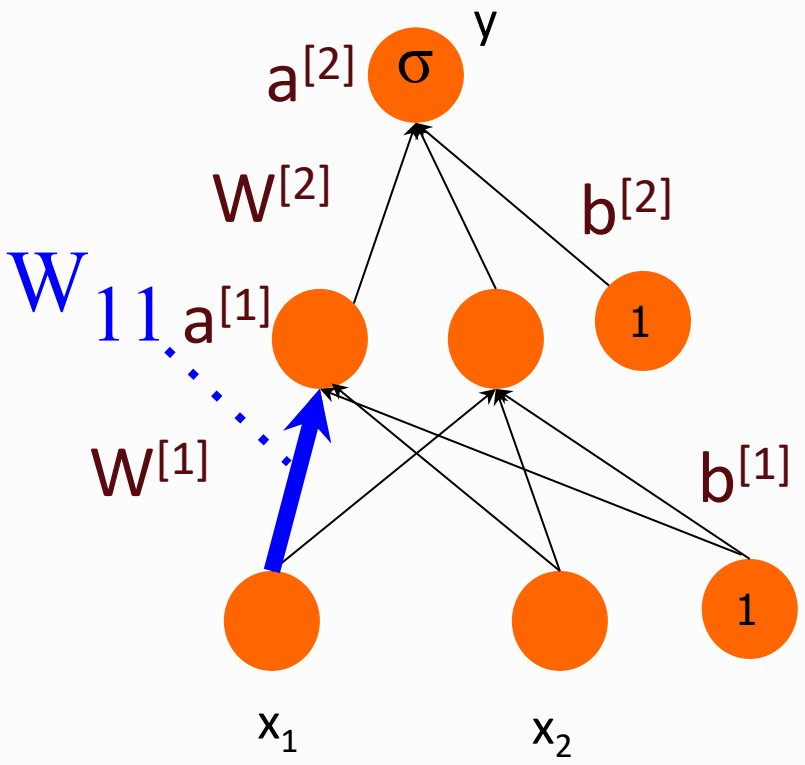
Backpropagation on a two layer network



$$
\begin{aligned}
z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\
a^{[1]} &= \text{ReLU}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

Slide adapted from Jurafsky & Martin

Backpropagation on a two layer network

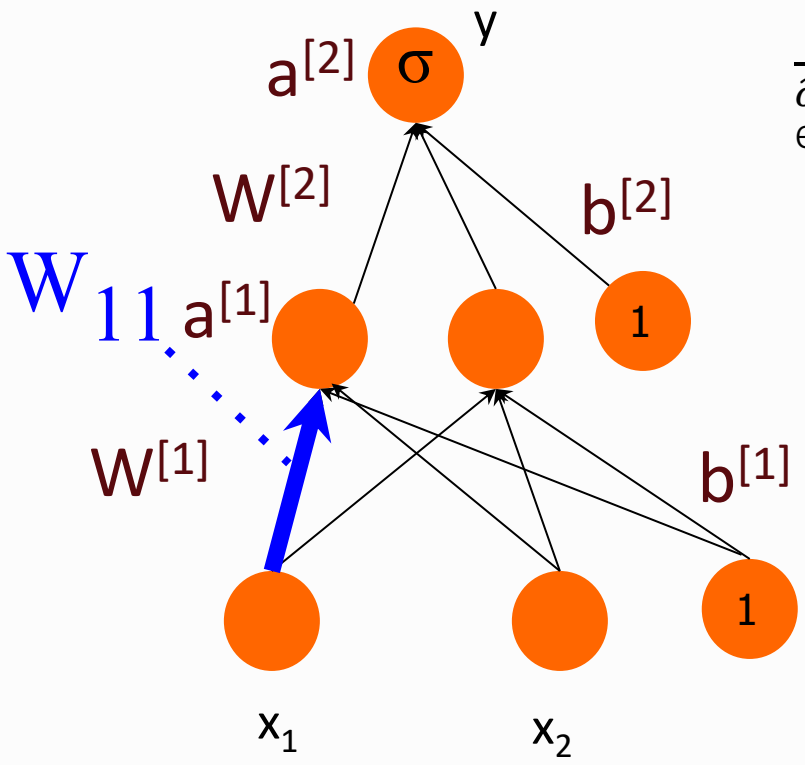


$$
\begin{aligned}
z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\
a^{[1]} &= \text{ReLU}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

$$
\frac{\partial L}{\partial W_{11}} = \frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial W_{11}^{[1]}}
$$

Slide adapted from Jurafsky & Martin

Backpropagation on a two layer network



$\frac{\partial L}{\partial a^{[2]}} = -\left(\frac{y}{a^{[2]}} + \frac{y-1}{1-a^{[2]}}\right)$ from derivative of cross-entropy loss

$\frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]}(1-a^{[2]})$ from derivative of sigmoid

$\frac{\partial z^{[2]}}{\partial a^{[1]}} = W_{11}^{[2]}$ from derivative of weighted sum

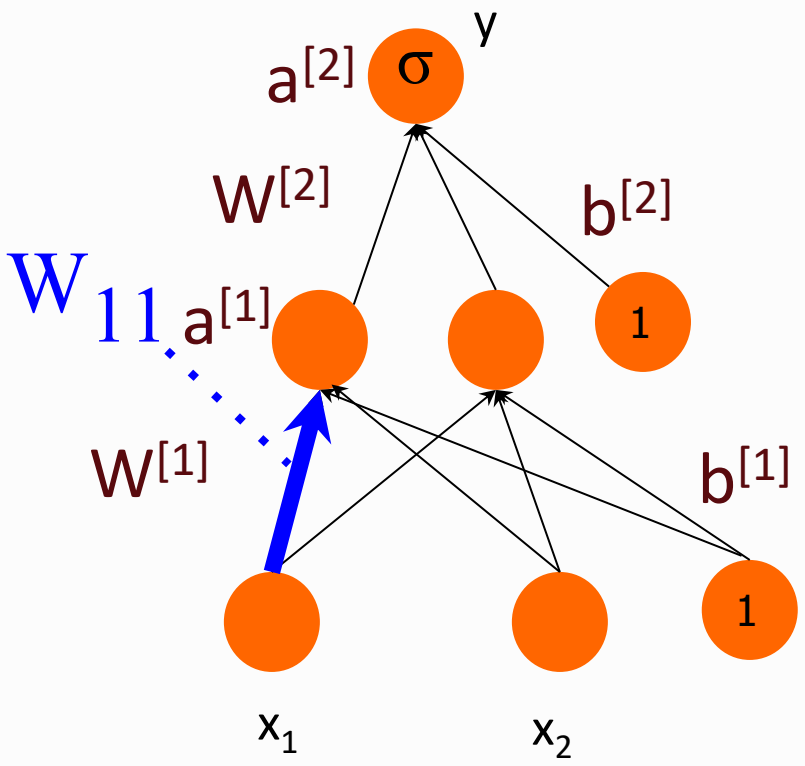
$\frac{\partial a^{[1]}}{\partial z^{[1]}} = \begin{cases} 0 \; for \; z^{[1]} < 0 \\ 1 \; for \; z^{[1]} \geq 0 \end{cases}$ from derivative of RELU

$\frac{\partial z^{[1]}}{\partial W_{11}^{[1]}} = x_1$ from derivative of product

$$\frac{\partial L}{\partial W_{11}} = \frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial W_{11}^{[1]}}$$

Slide adapted from Jurafsky & Martin

Backpropagation on a two layer network



$$\frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]} - y$$

$$\frac{\partial z^{[2]}}{\partial a^{[1]}} = W_{11}^{[2]} \text{ from derivative of weighted sum}$$

$$\frac{\partial a^{[1]}}{\partial z^{[1]}} = \begin{cases} 0 \; for \; z^{[1]} < 0 \\ 1 \; for \; z^{[1]} \geq 0 \end{cases} \text{ from derivative of RELU}$$

$$\frac{\partial z^{[1]}}{\partial W_{11}^{[1]}} = x_1 \text{ from derivative of product}$$

$$\frac{\partial L}{\partial W_{11}} = \frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial W_{11}^{[1]}}$$

Slide adapted from Jurafsky & Martin

Backpropagation on a two layer network

$y = 1$
$a^{[2]} = 2.5$
$W_{11}^{[2]} = 1.5$
$z^{[1]} = 3$
$x_1 = 4.2$

$$\frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]} - y$$

$$\frac{\partial z^{[2]}}{\partial a^{[1]}} = W_{11}^{[2]} \text{ from derivative of weighted sum}$$

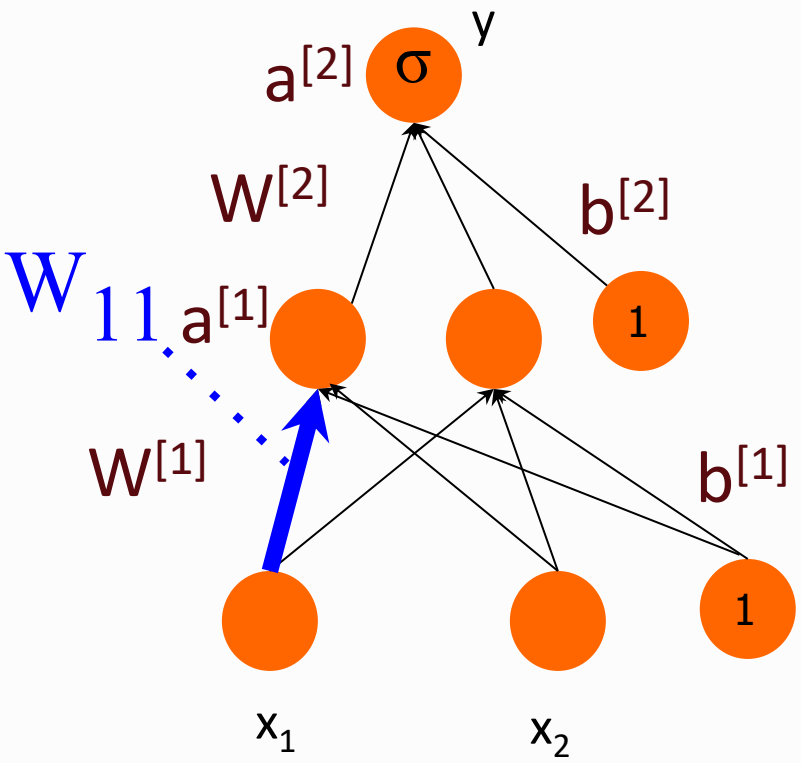
$$\frac{\partial a^{[1]}}{\partial z^{[1]}} = \begin{cases} 0 \; for \; z^{[1]} < 0 \\ 1 \; for \; z^{[1]} \geq 0 \end{cases} \text{ from derivative of RELU}$$

$$\frac{\partial z^{[1]}}{\partial W_{11}^{[1]}} = x_1 \text{ from derivative of product}$$

$$\frac{\partial L}{\partial W_{11}} = \frac{\partial L}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial W_{11}^{[1]}}$$

Slide adapted from Jurafsky & Martin

Backpropagation on a two layer network



$$\frac{\partial L}{\partial W_{11}} =$$

$$W_{11,t+1} =$$

Slide adapted from Jurafsky & Martin

Summary

For training, we need the derivative of the loss with respect to weights in early layers of the network

- But loss is computed only at the very end of the network!

Solution: **backpropagation**

Given the derivatives of all the functions in it we can automatically compute the derivative of the loss with respect to these early weights.

Muddiest point: Backprop is confusing!

With a neighbor, discuss what questions you have about backpropagation.

Coding activity: build an FFNN

Notebook: feedforward neural network

1. Go to this nbgitpuller link (also available on course website)
2. Start a server with **TEACH – 6 CPUs - 45 GB**
3. Load custom environment at **/ix1/cs1671-2026f/class-env/.pixi/envs/default**
4. Open **session14_ffnn.ipynb**